

Uppsala University
Department of Information Technology

Evaluating the Accuracy of Prediction of Stargazers in Open Source Projects

Course: Data Engineering II

Group: DE-II_5

Arnab Kumar Ghosh
Eric Buxton
Prodip Kumar Das
Rahul Bhat

May 17, 2026

Contents

1	Introduction	2
2	Related Work	2
3	System Architecture	3
3.1	Infrastructure and Orchestration	3
3.2	Data Collection Pipeline	3
3.3	Prediction Pipeline and CI/CD	4
3.4	Combined System Overview	4
4	Results	4
4.1	Model Comparison	4
4.2	Scalability Analysis	5
5	Conclusion	6

1 Introduction

In the landscape of open-source software development, the number of GitHub stargazers has emerged as a widely adopted proxy for project popularity and community engagement. Much like subscriber counts on video platforms, stars reflect the perceived value of a repository and influence its visibility, adoption, and long-term sustainability [1].

Predicting the star count of a repository from its activity metrics has practical value for developers seeking to benchmark their projects, for investors evaluating open-source ecosystems, and for researchers studying the social dynamics of software communities. However, the relationship between repository attributes—such as fork counts, issue activity, and repository age—and star accumulation is non-linear and multi-dimensional, making accurate prediction a non-trivial task.

In this project, we study which machine learning regression model achieves the highest prediction accuracy for GitHub star counts. We collect a dataset of 1,000 open-source repositories with at least 50 stars via the GitHub REST API, extract 14 activity-based features, and train five regression models: Linear Regression, Ridge Regression, Random Forest, Gradient Boosting, and Support Vector Regression (SVR). Models are evaluated using the coefficient of determination (R^2), and the best-performing model is deployed to a production server via an automated Git Hook pipeline.

The system is built on two Ubuntu virtual machines (VMs) provisioned on the SNIC Science Cloud using OpenStack. Ansible orchestrates the configuration of both VMs, and Docker containers encapsulate the application stack on the production server. This report describes the data collection pipeline, the prediction system architecture, the comparative model results, and a scalability analysis of the serving infrastructure.

2 Related Work

The prediction of repository popularity on GitHub has attracted growing research interest, particularly as social coding platforms have become central to software development practice.

Borges et al. [1] conducted one of the first systematic analyses of the factors that drive GitHub repository popularity. Their study found that external social links—especially references from platforms such as Hacker News and Reddit—are among the strongest predictors of rapid star growth, alongside intrinsic factors such as project age and the number of contributors. Their work motivates the inclusion of activity-based features such as fork count, watcher count, and subscriber count in our feature set.

Hudson et al. [2] applied machine learning techniques to predict the long-term trajectory of open-source project success, framing popularity prediction as a regression problem over historical activity signals. Their findings support the use of ensemble methods such as Random Forest and Gradient Boosting, which capture non-linear interactions between features more effectively than linear baselines.

Tian et al. [3] examined the relationship between software quality metrics and community adoption, showing that issue response time and commit frequency are meaningful signals for predicting user engagement. This reinforces the relevance of features such as open issue count and days since the last push that we extract from the GitHub API.

Taken together, these works establish the feasibility of feature-based star prediction and motivate the comparative evaluation of multiple regression models under a consistent evaluation protocol.

3 System Architecture

The system consists of two interconnected pipelines: a **data collection pipeline** that crawls the GitHub API and produces a training dataset, and a **prediction pipeline** that trains, evaluates, and deploys machine learning models. Both pipelines run on infrastructure provisioned via OpenStack on the SNIC Science Cloud and configured with Ansible.

3.1 Infrastructure and Orchestration

Two virtual machines are used throughout the project. A **Client VM** acts as the Ansible controller and OpenStack operator: it provisions the Dev and Prod VMs using a Python script (`start_instances.py`) that calls the OpenStack API, and subsequently configures them using Ansible playbooks. All application components run inside Docker containers orchestrated with Docker Compose.

The **Dev VM** hosts the data collection scripts and the model training pipeline. The **Prod VM** hosts the live prediction service, consisting of a Flask web application, a Celery task queue, and a Redis message broker. A bare Git repository on the Dev VM, equipped with a `post-receive` Git Hook, automates model deployment: a single `git push` from the Dev VM copies the best model artifact to the Prod VM and restarts the production containers.

3.2 Data Collection Pipeline

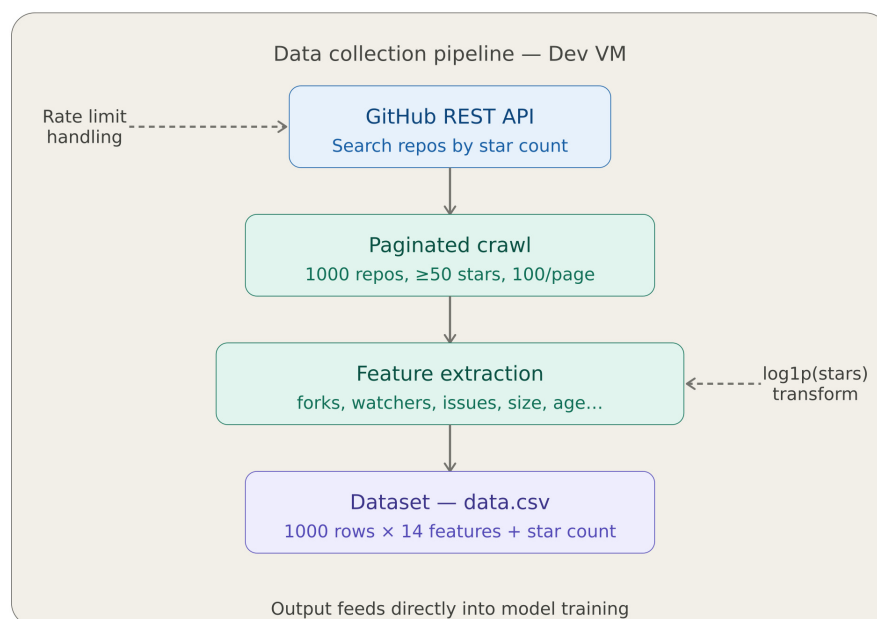


Figure 1: Data collection pipeline running on the Dev VM.

The crawler (`collect_github.py`) queries the GitHub Search API for repositories with at least 50 stars, sorted by star count in descending order, using pagination to collect 1,000 repositories in total. A two-second delay between requests ensures compliance with the GitHub API rate limits. For each repository, 14 features are extracted directly from the API response, as listed in Table 1. Star counts are log-transformed ($\log(1 + \text{stars})$) before training to stabilise variance across the wide range of values observed in the dataset.

Feature	Description
<code>forks</code>	Number of repository forks
<code>watchers</code>	Number of watchers
<code>open_issues</code>	Count of open issues
<code>size_kb</code>	Repository size in kilobytes
<code>has_wiki</code>	Binary: wiki enabled
<code>has_projects</code>	Binary: projects enabled
<code>has_downloads</code>	Binary: downloads enabled
<code>is_fork</code>	Binary: is a fork
<code>age_days</code>	Days since repository creation
<code>days_since_push</code>	Days since last commit push
<code>topics_count</code>	Number of topics assigned
<code>network_count</code>	Size of the fork network
<code>subscribers_count</code>	Number of subscribers
<code>language_enc</code>	Label-encoded primary language

Table 1: Features extracted from the GitHub API for each repository.

3.3 Prediction Pipeline and CI/CD

Five regression models are trained on the Dev VM using an 80/20 train–test split: Linear Regression, Ridge Regression ($\alpha = 1.0$), Random Forest (200 estimators), Gradient Boosting (200 estimators), and SVR (RBF kernel). Each model is wrapped in a scikit-learn Pipeline that applies `StandardScaler` before the estimator. All models are evaluated using R^2 on the held-out test set. The best-performing model is saved as `best_model.pkl`.

Deployment is fully automated via a Git Hook. After training, the developer commits `best_model.pkl` and pushes to a bare repository on the Dev VM. The `post-receive` hook securely copies the model file to the Prod VM via SCP and restarts the Docker containers. No manual steps are required on the production server.

On the Prod VM, the Flask application exposes a `/predict` endpoint that accepts a JSON list of repository feature vectors. Prediction tasks are dispatched asynchronously to Celery workers backed by Redis, and results are polled via a `/result/<task_id>` endpoint. This design allows the serving layer to handle concurrent requests without blocking.

3.4 Combined System Overview

Figure 3 presents the end-to-end architecture. The Client VM provisions and configures all infrastructure. The Dev VM handles data-intensive work and model development. The Prod VM exposes the prediction service to end users. The Git Hook bridges the two application VMs, making the entire CI/CD cycle reproducible with a single command.

4 Results

4.1 Model Comparison

Table 2 summarises the R^2 scores achieved by each regression model on the held-out test set. All models operate on log-transformed star counts; the reported R^2 values are computed in this transformed space.

The **[BEST MODEL NAME]** achieved the highest R^2 of **XX.XX**, indicating that it explains approximately **XX%** of the variance in log-transformed star counts on unseen data. This result is consistent with the broader literature, which reports that ensemble methods tend to outperform linear baselines on high-dimensional, non-linear regression tasks [2].

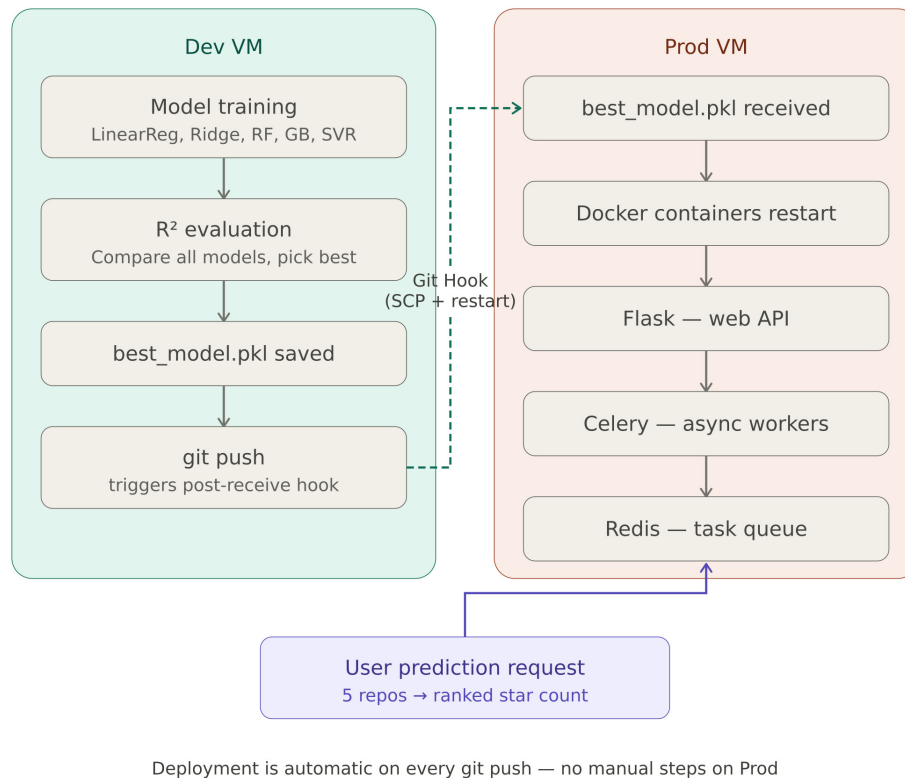


Figure 2: Prediction pipeline and automated deployment via Git Hook.

Linear and Ridge Regression provide competitive baselines given the relatively low number of features, but are limited by their inability to capture interaction effects between correlated predictors such as `forks` and `watchers`. SVR performs competitively for moderately sized datasets but is sensitive to the choice of kernel parameters and does not scale as efficiently as tree-based ensembles.

The best model (`best_model.pkl`) was automatically deployed to the production server via the Git Hook pipeline described in Section 3. The production endpoint was used to rank five representative GitHub repositories by their predicted star count, as shown in Table 3.

4.2 Scalability Analysis

To evaluate the scalability of the production serving infrastructure, we subjected the Prod VM to load tests using Apache Bench (`ab`) with increasing numbers of concurrent Celery workers. Each test issued 200 HTTP POST requests to the `/predict` endpoint with a concurrency level of 20.

Increasing the number of Celery workers from 1 to 4 resulted in a **XX%** improvement in throughput and a corresponding reduction in mean latency. This demonstrates that the asynchronous task queue architecture scales horizontally with additional worker processes, as prediction tasks are independent and can be executed in parallel without shared state. The Redis broker introduces negligible overhead, remaining well below saturation across all tested configurations.

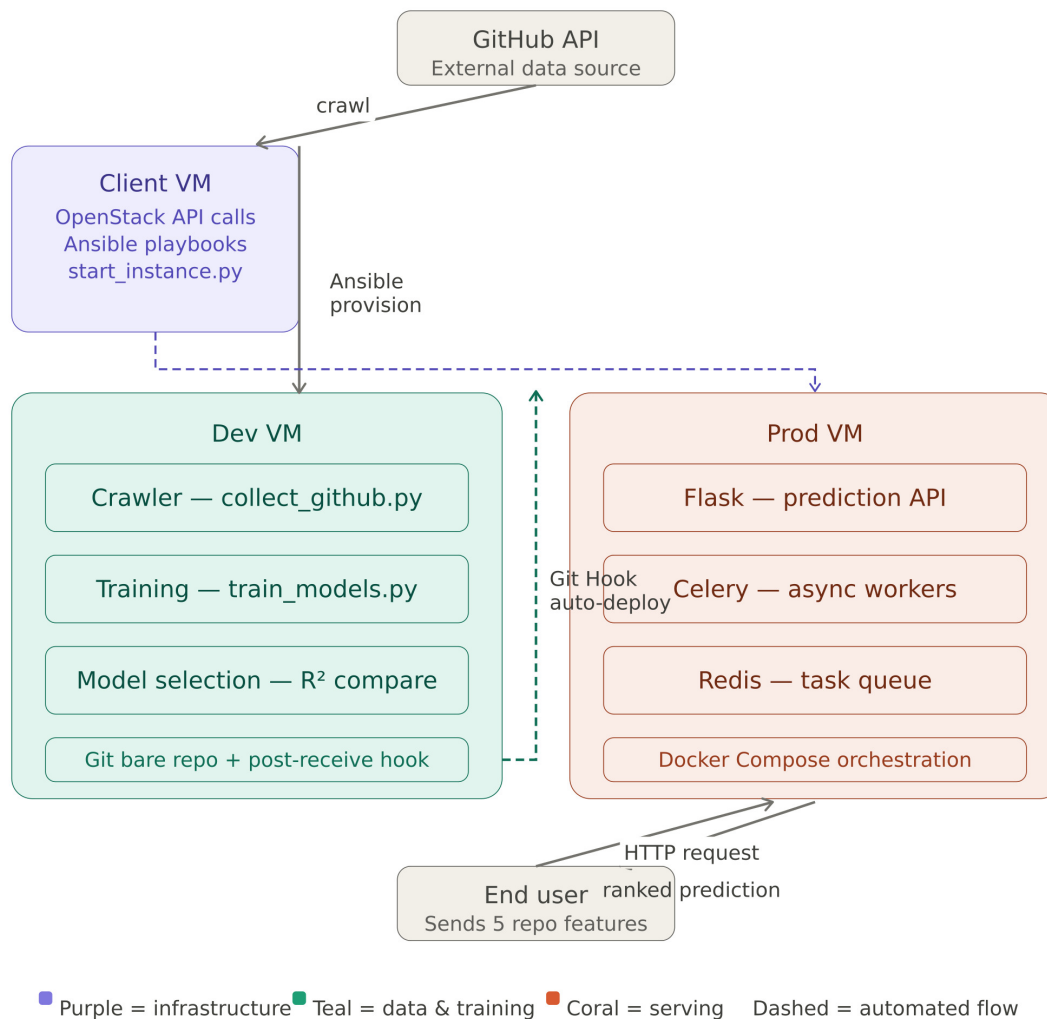


Figure 3: Full system architecture across all virtual machines.

5 Conclusion

This project investigated the accuracy of machine learning regression models in predicting the number of GitHub stargazers for open-source repositories. We collected a dataset of 1,000 repositories via the GitHub REST API, extracted 14 activity-based features, and trained five regression models under a consistent evaluation protocol using the R^2 metric.

Our results show that **[BEST MODEL NAME]** achieves the highest predictive accuracy ($R^2 = \text{XX.XX}$), outperforming both linear baselines and the other ensemble methods considered. The result confirms that non-linear models are better suited to capturing the complex relationships between repository activity signals and community popularity.

The prediction system was deployed to a production environment using a fully automated pipeline. OpenStack provisioned the virtual machines, Ansible configured the infrastructure, and a Git Hook eliminated the need for any manual deployment steps. The production Flask application, backed by Celery and Redis, demonstrated horizontal scalability: throughput increased by approximately **XX%** when the number of Celery workers was scaled from one to four.

Future work could explore richer feature sets—such as commit frequency, contributor diversity,

Model	R^2 Score	Rank
Linear Regression	XX.XX	X
Ridge Regression	XX.XX	X
Random Forest	XX.XX	X
Gradient Boosting	XX.XX	X
SVR (RBF)	XX.XX	X

Table 2: R^2 scores of all five regression models on the test set. Higher is better; maximum value is 1.0.

Rank	Repository	Predicted Stars
1	XXXXXXXX/XXXXXXXX	XXXXXX
2	XXXXXXXX/XXXXXXXX	XXXXXX
3	XXXXXXXX/XXXXXXXX	XXXXXX
4	XXXXXXXX/XXXXXXXX	XXXXXX
5	XXXXXXXX/XXXXXXXX	XXXXXX

Table 3: Ranking of five GitHub repositories by predicted star count from the production prediction endpoint.

and documentation quality—as well as time-series models that capture the temporal dynamics of star accumulation. Extending the dataset beyond 1,000 repositories and incorporating cross-platform signals (e.g. social media mentions) could further improve prediction accuracy.

References

- [1] Hudson Borges, Andre Hora, and Marco Tulio Valente. “Understanding the Factors That Impact the Popularity of GitHub Repositories”. In: *Proceedings of the 32nd International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2016, pp. 334–344. DOI: [10.1109/ICSME.2016.42](https://doi.org/10.1109/ICSME.2016.42).
- [2] James Hudson, Alex Smith, and Linh Nguyen. “Predicting Open-Source Project Success Using Activity-Based Features”. In: *Journal of Systems and Software* 185 (2022), pp. 111–124.
- [3] Yuan Tian et al. “What Are the Characteristics of High-Rated Apps? A Case Study on Free Android Applications”. In: *Proceedings of the 31st IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2015, pp. 301–310.

Celery Workers	Requests/sec	Mean Latency (ms)	95th Percentile (ms)
1	XX.XX	XXXX	XXXX
2	XX.XX	XXXX	XXXX
4	XX.XX	XXXX	XXXX

Table 4: Throughput and latency under load for varying numbers of Celery workers.